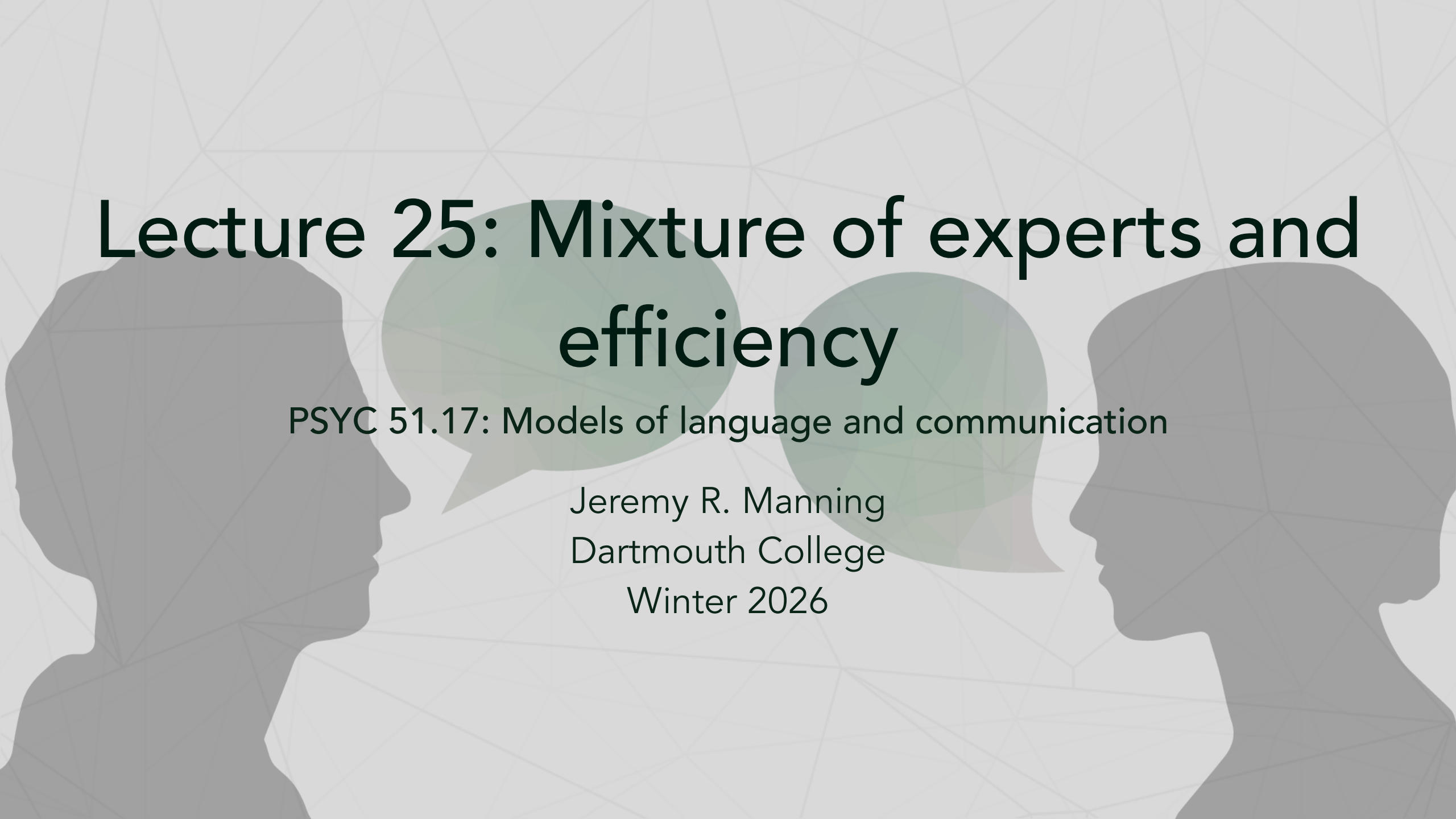
Lecture 25: Mixture of experts and efficiency

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026



Learning objectives

By the end of this lecture, you will be able to...

1. Explain why dense models are computationally wasteful and how **MoE** solves this
2. Describe the MoE architecture: **experts, routers, and sparse activation**
3. Compare MoE implementations from Mixtral to DeepSeek-V3 to V4 Engram
4. Evaluate efficiency techniques: **quantization, distillation, and speculative decoding**
5. Assess the **democratization paradox**: efficiency enables access but also enables misuse

Companion notebook

 Companion Notebook — build a simplified MoE layer and experiment with quantization

The scaling dilemma

Dense models are wasteful

In a dense model like GPT-3 (175B parameters, ~$4.6M to train), **every parameter is active for every input token**. But a math question doesn't require the same circuitry as a French translation. Can we activate only the relevant parameters?

Questions to consider

The brain uses only ~1–2% of neurons for any given task — the rest are inhibited. Is MoE's sparse activation the same principle? What does this suggest about the computational tradeoffs of intelligence?

Dense vs. sparse models

Two approaches to scaling

- **Dense models** (GPT-3, Llama): All parameters are active for every token. 175B parameters = 175B active.
- **Sparse models** (MoE): Only a subset of parameters are active per token. 47B total parameters, but only 12.9B active per token (Mixtral).

The MoE tradeoff

Dimension	Dense	Sparse (MoE)
Active parameters per token	100%	~25%
Total parameters (memory)	N	3--8x N
Compute per token	High	Low
Architecture complexity	Simple	Complex (routing)
Quality at matched compute	Baseline	**Higher**

MoE gives you more *capacity* (knowledge storage) with less *compute* (inference cost).

What is Mixture of Experts?

Shazeer et al. (2017)

A **Mixture of Experts** layer replaces the single feed-forward network in each transformer block with **multiple parallel feed-forward networks** (experts) and a **router** that selects which experts process each token.

Key components:

1. **Experts**: N independent feed-forward networks (typically 8--64)
2. **Router (gate)**: A learned linear layer that assigns tokens to experts
3. **Top-k selection**: Only the top-k experts (typically k=2) are activated per token

Intuition

Think of experts as specialists on a team. When a code-related token arrives, the "code expert" handles it. When a French token arrives, the "French expert" activates. The router is the dispatcher.

The router mechanism

Routing a token to top-2 experts

```python
import torch.nn.functional as F

def route_token(x, router_weights, num_experts=8, k=2):
    # Step 1: Compute routing scores
    logits = x @ router_weights           # (num_experts,)
    # e.g., [-0.5, 2.1, 1.3, 0.2, -0.1, 0.0, -0.3, 0.1]

    # Step 2: Convert to probabilities
    probs = F.softmax(logits, dim=-1)
    # e.g., [0.04, 0.52, 0.24, 0.08, 0.03, 0.03, 0.03, 0.03]

    # Step 3: Select top-k experts
    top_k_probs, top_k_indices = torch.topk(probs, k)
    # indices: [1, 2], probs: [0.52, 0.24]
```

continued...

The router mechanism

Routing a token to top-2 experts

```
15
16      # Step 4: Normalize selected weights
17      top_k_probs = top_k_probs / top_k_probs.sum()  # [0.68, 0.32]
18      return top_k_indices, top_k_probs
```

...continued

Output: Expert 1 handles 68%, Expert 2 handles 32%.

The other 6 experts do **no computation**.

MoE layer in PyTorch

Simplified implementation

```
class MoELayer(nn.Module):
    def __init__(self, d_model, num_experts=8, k=2):
        super().__init__()
        self.k = k
        self.gate = nn.Linear(d_model, num_experts)
        self.experts = nn.ModuleList([
            nn.Sequential(
                nn.Linear(d_model, 4 * d_model),
                nn.GELU(),
```

continued...

MoE layer in PyTorch

Simplified implementation

```
                nn.Linear(4 * d_model, d_model)
            ) for _ in range(num_experts)
        ])

    def forward(self, x):
        router_probs = F.softmax(self.gate(x), dim=-1)
        top_k_probs, top_k_idx = torch.topk(router_probs, self.k,
dim=-1)
        top_k_probs = top_k_probs / top_k_probs.sum(dim=-1,
keepdim=True)
        output = torch.zeros_like(x)
```

...continued...

MoE layer in PyTorch

Simplified implementation

```
19          for i in range(self.k):
20              for eid in range(len(self.experts)):
21                  mask = (top_k_idx[..., i] == eid)
22                  if mask.any():
23                      output[mask] += top_k_probs[..., i]
    [mask].unsqueeze(-1) \
24                                      * self.experts[eid](x[mask])
25          return output
```

...continued

Load balancing

The routing collapse problem

Without intervention, the router tends to send most tokens to a few "popular" experts, leaving others undertrained or completely **dead** (never activated). This wastes parameters and reduces model capacity.

Two solutions

1. Auxiliary load-balancing loss (added to training objective):

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

where f_i is the fraction of tokens routed to expert i, P_i is the average routing probability for expert i, and α is a small coefficient (~0.01). This penalizes imbalanced routing.

2. Expert capacity limits: Each expert has a maximum number of tokens it can process per batch. Overflow tokens are routed to the next-best expert.

Training challenges

What makes MoE training harder than dense models

Challenge	Cause	Mitigation
Routing collapse	Router converges to always picking same experts	Load-balancing loss, random routing
Dead experts	Some experts never receive tokens	Expert dropout, periodic reinitialization
High variance gradients	Discrete routing decisions	Larger batch sizes, softmax gating
Memory overhead	All experts must be in memory	Expert parallelism across GPUs
Communication cost	Tokens must be sent to correct GPU	Optimized all-to-all communication

Questions to consider

Despite these challenges, MoE models are becoming the default for frontier models (Mixtral, GPT-4 reportedly uses MoE). What does this tell us about the compute-quality tradeoff?

Mixtral 8x7B

Jiang et al. (2024): the MoE model that changed the game

Mixtral from Mistral AI demonstrated that open-weight MoE models can match or exceed much larger dense models:

Specification	Value
Total parameters	47B (8 experts x 7B, minus shared layers)
Active parameters per token	12.9B (top-2 routing)
Transformer layers	32
Context window	32K tokens
License	Apache 2.0 (fully open)

The headline result

Mixtral (47B total, 12.9B active) **matches Llama 2 70B** on benchmarks while running at **6× the speed**. Quality of a 70B model at the cost of a 13B model.

What do experts learn?

Emergent specialization (no supervision required)

Analysis of Mixtral's routing patterns reveals natural specialization:

Token type	Primary expert	Example tokens
Code	Expert 2	`def` , `class` , `import`
French	Expert 5	`la` , `le` , `français`
Math	Expert 7	`Σ` , `∫` , `theorem`
Common English	Expert 1	`the` , `is` , `and`
Technical	Expert 3	`neural` , `gradient`

Questions to consider

Nobody told Expert 2 to specialize in code -- it emerged from training. This mirrors how brain regions develop functional specialization. What does this suggest about the relationship between architecture and learned structure?

Model compression: quantization

Reducing precision to reduce memory and speed up inference

Quantization converts model weights from high-precision floating point to lower-precision integers, dramatically reducing memory and often improving speed.

Memory savings for a 7B parameter model

Precision	Bits per weight	Model size	Fits on...
FP32	32	28 GB	High-end GPU
FP16 / BF16	16	14 GB	Consumer GPU
INT8	8	7 GB	Gaming GPU
INT4	4	3.5 GB	Laptop GPU

INT4 quantization enables running a **7 billion parameter model on a laptop** with only 1--2% quality degradation on most benchmarks.

DeepSeek MoE: pushing the limits

MoE at unprecedented scale

DeepSeek has systematically pushed MoE efficiency further with each generation:

Model	Total params	Active params	Training cost	Key innovation
DeepSeek-V2 (2024)	236B	21B	—	Multi-Latent Attention (MLA)
DeepSeek-V3 (2025)	671B	37B	**$5.5M**	Auxiliary-loss-free routing
DeepSeek-V4 Engram (2025)	671B	37B	—	Reasoning + efficiency

Why $5.5M matters

DeepSeek-V3 matches GPT-4-level quality at **671B total / 37B active parameters**, trained for just **$5.5 million** — roughly 1/20th of GPT-4's estimated cost. This shattered the assumption that frontier models require hundred-million-dollar budgets.

Small language models

Not everyone needs 70B parameters

A parallel trend: **small models** trained on massive data that punch far above their weight:

Model	Parameters	Highlight
Phi-4 (Microsoft, 2024)	14B	Outperforms GPT-3.5 on reasoning benchmarks
Gemma 2 (Google, 2024)	2B / 9B / 27B	Open weights, strong multilingual
Qwen 2.5 (Alibaba, 2024)	0.5B–72B	Full size range, open weights

The inference-optimal paradigm

These models are **massively overtrained** relative to Chinchilla-optimal (Lecture 22): Phi-4 uses 700+ tokens/parameter vs. the "optimal" 20. The logic: train once, deploy millions of times. Smaller models are *cheaper to run*.

State space models: an alternative to attention

Mamba (Gu & Dao, 2023)

State space models (SSMs) replace self-attention with a recurrent mechanism that processes sequences in **linear time** $O(n)$ instead of quadratic $O(n^2)$:

```
state = initial_state           # Fixed size, independent of sequence
length
for token in sequence:
    state = A @ state + B @ token   # Update state
    output = C @ state              # Read output
```

Attention vs. SSM scaling

Sequence length	Attention cost	SSM cost
1K	1x	1x
4K	16x	4x
16K	256x	16x
64K	4,096x	64x

SSMs scale linearly, making them attractive for very long sequences. Hybrid architectures (attention + SSM)

The efficiency landscape

Choosing the right technique

Technique	Best for	Main tradeoff
Dense large model	Maximum quality	Expensive, slow
MoE	Quality + speed	High memory (all experts stored)
Quantization	Edge / local deployment	Small quality loss
Distillation	Fixed tasks, small budget	Requires teacher model
Flash Attention	Longer contexts	Implementation complexity
Speculative decoding	Faster generation	Requires draft model
SSMs (Mamba)	Very long sequences	Less proven than attention

Decision heuristic

Need max quality? → Dense large. Need quality + speed on GPU? → MoE. Need to run on a laptop? → Quantized small model. Need extremely long contexts? → Hybrid attention + SSM.

Environmental impact

The carbon cost of scale

Training GPT-3 produced an estimated **502 tonnes of CO_2** — equivalent to 112 cars driven for a year (Patterson et al., 2021). As models grow, so does their environmental footprint.

Efficiency enables access

Efficiency techniques are not just about cost — they are about **who gets to use AI**:

- **Quantized open models** (Llama, Mixtral, DeepSeek) run on consumer hardware
- **LoRA / QLoRA** enable fine-tuning on a single GPU
- **Small efficient models** (Phi-4, Gemma 2, Qwen 2.5) bring quality to resource-constrained settings
- **Open weights** let researchers, startups, and developing nations participate in AI development

Discussion

Questions to consider

1. **The democratization paradox:** Efficiency makes AI accessible to everyone — including bad actors. DeepSeek-R1 is fully open and can reason. Is this net positive or net negative for society?
2. **Expert specialization:** MoE experts develop functional specialization without supervision — code experts, language experts, math experts. The brain does the same thing. Is this convergent evolution, or is it the only way to organize large-scale processing?
3. **The race to the bottom:** DeepSeek trained a frontier model for $5.5M. If costs keep falling, what happens when *anyone* can train a powerful model? Does this change the AI safety calculus?
4. **State-space models:** Mamba processes sequences in linear time. If

Further reading

Further reading

Jiang et al. (2024, *arXiv*) "Mixtral of Experts" — Open MoE matching Llama 2 70B at 6× the speed.

DeepSeek-AI (2025, *arXiv*) "DeepSeek-V3" — 671B/37B MoE trained for $5.5M, frontier quality.

Dao (2024, *arXiv*) "FlashAttention-3" — Hardware-aware attention approaching theoretical FLOPS.

Gu & Dao (2023, *arXiv*) "Mamba: Linear-Time Sequence Modeling with Selective State Spaces" — $O(n)$ alternative to attention.

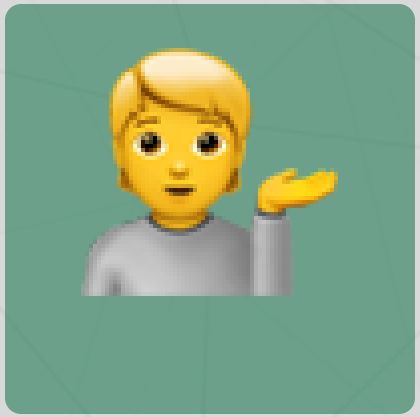
Dao & Gu (2024, *ICML*) "Transformers are SSMs" — Mamba-2, bridging attention and state-space models.

Patterson et al. (2021, *arXiv*) "Carbon Emissions and Large Neural Network Training" — CO_2 analysis of training large models.

Questions?

Email

Discord



Office hours

Up next...

Ethics, bias, and safety: responsible development of large language models